

Sprint 3 Dokumentation P2P Kompetenztausch Plattform

Team Kompetenztausch

9. Juni 2026

Qualität und Konsolidierung

Im Sprint 3 wurden mehrere funktionale und strukturelle Schwächen im Backend erkannt. Besonders bei Match Requests, Sessions und Ratings musste klarer abgegrenzt werden, welches Modul welche Verantwortung übernimmt. Ein Match Request beschreibt nur die Anfrage zwischen zwei Nutzern, während die SkillSession den tatsächlichen Kompetenztausch nach einer angenommenen Anfrage abbildet. Die Punktevergabe wurde deshalb nicht direkt an User, Chat oder Conversation gekoppelt, sondern an eine konkrete Session. Dadurch wurde die fachliche Grundlage für Bewertungen deutlich sauberer.

Auch die bestehende Skill-Suche wurde konsolidiert und erweitert. Die bisherige Suche erkannte vor allem exakte oder teilweise Übereinstimmungen. Durch die embedding-basierte semantische Suche können nun auch inhaltlich ähnliche Skills berücksichtigt werden, zum Beispiel `backend` mit `Java`, `Spring Boot` oder `REST API`. Zusätzlich werden sowohl `offeredSkills` als auch `wantedSkills` unterstützt. Darauf aufbauend wurde ein Discover-System umgesetzt, das beide Richtungen des Kompetenztauschs bewertet und besonders passende Two-Way-Matches höher gewichtet. Die Gemini-Integration wurde über Port/Adapter-Strukturen umgesetzt. API-Keys werden nicht hardcoded, sondern über `GEMINI_API_KEY` gesetzt; raw Embeddings werden nicht im REST-Response zurückgegeben.

Im Bereich Match Request und Admin-Rollen wurden ebenfalls Schwächen bereinigt. Dazu gehörten uneinheitliche Fehlerbehandlung, offene Testlücken bei rollenbasierten Zugriffen und unklare Admin-Sonderfälle. Verbessert wurden unter anderem der Schutz vor Selbstanfragen, und die Trennung von USER- und ADMIN-Rechten über `@RolesAllowed`.

Im Frontend wurde In diesem Sprint wurde das in den Vor-Sprints noch unvollständige Matching-System vollständig konsolidiert. Logikfehler in der Suche wurden behoben, sodass der eigene Nutzer sowie bereits gematchte Profile nicht mehr fälschlicherweise angezeigt werden. Der gesamte MatchingProzess wurde auf ein durchdachtes Request-System umgestellt (Anfrage senden, ausstehend, akzeptiert), ergänzt durch einen dedizierten Reiter zur Verwaltung von Kontaktanfragen. Zudem wurde eine große Lücke im Frontend geschlossen: die Integration eines zentralen Admin-Screens. Dieses neue Verwaltungs-UI ermöglicht eine sichere, rollenbasierte (USER vs. ADMIN) Steuerung des Systems direkt aus der App heraus. Begleitet wurden diese funktionalen Konsolidierungen von einer optischen Modernisierung durch konsistente UI-Komponenten.

Code- und Prozessqualität

Die neuen Backend-Funktionen wurden entlang der bestehenden hexagonalen Architektur umgesetzt. REST-Resources bleiben möglichst schlank und delegieren an Use Cases beziehungsweise Application Services. Die Business-Logik liegt in Services und Domain-Modellen, wodurch Validierungen und Statusübergänge zentraler behandelt werden konnten.

Beim Rating- und Session-Flow wurde besonders auf klare Zuständigkeiten geachtet. Die Session verwaltet den fachlichen Zustand des Kompetenztauschs, zum Beispiel `ACTIVE`, `RATING_OPEN` und `RATING_CLOSED`. Das Rating-Modul prüft, ob eine Bewertung erlaubt ist, speichert Bewertungen zunächst als `PENDING` und veröffentlicht sie später als `PUBLISHED`. Nur veröffentlichte Bewertungen werden für die durchschnittliche Bewertung und die Anzahl der Bewertungen eines Nutzers berücksichtigt. Diese `RatingSummary` wird anschließend im User-Profil und in der Matching-Suche angezeigt.

Auf Persistenzebene wurden neue Flyway-Migrationen für Match Requests, Skill Sessions, Skill Embeddings und Ratings ergänzt. Dabei musste die Reihenfolge der Migrationen wegen Foreign-Key-Abhängigkeiten sauber abgestimmt werden. Außerdem wurden bestehende Tests angepasst, da neue Use Cases und Parameter mehrere Methodensignaturen verändert haben. Einige ältere REST-Tests wurden überarbeitet, weil direkte Assertions auf HTTP-Statuscodes wie 400 oder 404 stabiler sind als erwartete Exceptions.

Zum Abbau technischer Schulden im Frontend erfolgte ein tiefgreifendes Architektur-Refactoring mit `flutter_riverpod`. Die enge Kopplung zwischen Screens und Services wurde durch eine saubere Dependency Injection (DI) abgelöst. Der globale Zustand (Token, Login, Rolle) wird nun zentral und reaktiv verwaltet, was unübersichtliche `setState`-Aufrufe minimiert und die hexagonale Architektur des Backends im Frontend widerspiegelt. Die Code-Stabilität wurde durch gezielte Provider-Unit-Tests gestärkt, die die korrekte Initialisierung der Services und den sicheren Logout-Prozess zuverlässig absichern.

Teamreflexion

Die Zusammenarbeit und Kommunikation im Team verliefen in diesem Sprint reibungslos und hochgradig strukturiert. Die organisatorischen Probleme aus den vorherigen Sprints wurden direkt im Anschluss an Sprint 2 offen diskutiert, was sich extrem positiv auf den aktuellen Sprint auswirkte. Der GitLab-Workflow wurde dieses Mal akribisch und diszipliniert umgesetzt: Vor Entwicklungsbeginn erfolgte eine klare Aufgabenverteilung und die Erstellung aller zugehörigen Issues. Jeder Merge Request wurde nach Abschluss eines Issues sorgfältig kontrolliert und gereviewt, bevor er in den Main-Branch integriert wurde. Dadurch konnten bisherige Koordinationsschwierigkeiten vollständig vermieden werden.

KI-Reflexion

KI war in Sprint 3 vor allem bei Strukturierung, Debugging, Boilerplate-Code, Testanpassungen und Dokumentationsentwürfen hilfreich. Besonders nützlich war KI bei Architekturfragen, zum Beispiel bei der Abgrenzung zwischen Match Request, Session, Conversation und Rating. Auch bei Quarkus-, JPA- und Flyway-Problemen sowie bei der Erstellung eines vollständigen Postman-Testflows konnte KI die Analyse beschleunigen.

Besonders hilfreich war KI bei der fachlichen Vorüberlegung, wie wir später mit Punktevergaben in Disputes umgehen können. Dieser Teil wird erst im nächsten Sprint umgesetzt. Außerdem unterstützte KI bei der Strukturierung der Suchlogik, bei der Ableitung relevanter Testfälle und bei der klaren Trennung zwischen Sessions, Matching, Matching Requests und Rating.

Auch in Frontend KI wurde in diesem Sprint gezielt als beratendes Werkzeug genutzt. Bei der architektonischen Integration von Riverpod diente sie zur Ideenfindung und der Evaluierung von Best Practices. Zudem half die KI bei der Fehlerbehebung, indem sie Vorschläge für sinnvolle Debug-Ausgaben im Code generierte, um Fehlerquellen schneller zu identifizieren.